

COMP.2560: System Programming

LAB #2: Scripts and Debugging

LAB 2: Overview

The main purpose of this lab is to:

- Learn how to use the “script” and “scriptreplay” commands to record and replay a terminal session,
- Learn the basics of debugging using gdb, and
- Practice using gdb to debug a program.

There are three parts in this lab, described below.

LAB 2 – Part I: The script and scriptreplay commands

We showed you in Lab 1 how to record your whole computer screen. In fact, the Unix/Linux system itself has two commands that could be used to “record” your terminal session (not the whole computer screen) and replay it. Note it only records your active terminal window, not the whole screen of your machine. These commands were traditionally used in the past to demonstrate how things are done and to prove you have done it yourself, among other purposes.

The “script” and “scriptreplay” are very useful Unix/Linux commands to record and replay a terminal session. The recorded terminal session is saved in a text file and, optionally, together with an associated timeline file so that you can replay your terminal session later for demonstration and other purposes. **Most importantly in this course, the recorded terminal session will *sometimes* be required to be submitted. Therefore, it is very important you know how to use these two commands.**

Watch the followings video for the typical use of the “script” and “scriptreplay” commands in Linux: [The Linux 'script' Command: Record Your Shell Sessions](#).

This video shows a few ways to use the script command. Pay attention to the way script/scriptreplay command is used from 2:44s to 4:40s. That is what you will be using them for this semester, if asked. In short, if you want to record your active terminal window, type the following:

```
$ script nameofScriptFile.log --timing=time.log
```

When you are done recording everything, press “Ctrl+d” and the script command will terminate. The name of the script file produced, shown above as “nameofScriptFile.log”, is of your choice and the extension is normally “.log”.

The “--timing=time.log” portion of the command shows that the timing information of your terminal activities are recorded in the file called “time.log”, again, you could use any file name you like for the timing file. You could simply understand the way how script command works as the script file “nameofScriptFile.log” records all the interactions between you and the shell, and the timing file “time.log” records the exact time those interactions happened.

What you recorded using the script command above can be replayed. For example, type the following:

```
$ scriptreplay -s NameofScriptFile.log --timing=time.log
```

You could also check out the manual page for “script” and “scriptreplay” by typing:

```
man script  
man scriptreplay
```

at your shell prompt if you want to know more about these two commands.

LAB 2 – Part 2: GDB – The tool to debug your program

GDB is a commonly used, very useful program to debug almost any program under Linux/Unix. There are many excellent resources online explaining how to use gdb. To get you started quickly, we have chosen a very short tutorial that covers the essentials of gdb. Please follow this link:

[Quick Gdb Guide](#)

and read the whole page.

The document was written for debugging C++ programs. All the commands introduced in the document still apply to debugging C programs. The only two differences we need to keep in mind are:

- a) In the document, in “Preparation”, step 1, it shows the command to compile a C++ program is:

```
g++ -g printch.cpp -o printch
```

Note in the above command that “printch.cpp” is the name of the C++ source code, and “-o printch” specifies the name of resulting executable file. If you do not specify the name of the output executable file, then it is named “a.out” by default.

Since we are coding C programs, you need to change the above command to:

```
gcc -g sourcecode -o executable_name, or, simply:
```

```
gcc -g sourcecode (in this case, a.out will be the name of the resulting executable file by default).
```

- b) In the document, the command to run the gdb is as follows:

```
gdb -x gdb -tui printch
```

Note that “printch” is the name of the executable file.

On our CS Linux servers, the “-x gdb” option is not supported, so use just use the following command:

```
gdb --tui printch
```

LAB 2 – Part 3: Using the GDB program

Now go ahead and download the source code found in file “`copy.c`”, attached with this lab handout. This program, when working correctly, will copy a file and its usage is as

```
Shell prompt> ./copy source dest  
(assuming the executable files is named copy)
```

For example, the following command:

```
Shell prompt> ./copy copy.c copy.bak
```

will make a copy of the file “`copy.c`” and the copy is called “`copy.bak`”

The program “`copy.c`” (please note this version of “`copy.c`” is slightly different than the one we discussed in class) attached could be successfully compiled, but when run, it does not do what it is supposed to do.

Use GDB to launch a debugging session to fix the problem.

LAB 2 – Submission

Record a brief and to the point video with audio to show how you use GDB to debug the `copy.c` program. Explain every step in your debugging session, as well as what problems you located.

To formally submit your work for this Lab:

- Upload your recorded video file to any free video hosting site such as YouTube or Microsoft Stream (included in Office 365 suite, free to students)
- Submit the shareable link to the video file you created in Brightspace.
Make sure you provide appropriate access permissions.
- Submit a text file, briefly explaining what the bug(s) is in the original `copy.c` program.
- When submitting, please indicate in the submission textbox the Lab section you belong to.

Evaluation:

1. It is our intention to evaluate your work being done in-person, while you are in the lab.
2. It is strongly suggested that you attend in-person to familiarize yourselves with the Lab infrastructure, meet with the GATAs and solve problems.
3. Each session is only 80 minutes long, so it is strongly suggested that you arrive early, are focused and do not disrupt the process.
4. It is desired that you complete your work while in the lab and get graded after live review from the GATAs.
5. Should you need more time, you can take your work home and present it to the GATAs on the very next lab session (the week following).
6. Even if you decide to work from home and submit your work remotely, you still need to have your work peer reviewed.